# Crypto – TME – Notes

## FLAG 1 : Déchiffrement symétrique

### Objectif :

- Déchiffrer un Ciphertext en un Plaintext avec un Password.

### Méthode :

- Lancer le code « decrypt_ciphertext_post_it.py » qui utilise le fonction « decrypt() » du code « ./PrimaryFunctions/decrypt.py » avec en entrée le Ciphertext et le Password.

### Explication du code :

- Convertit le Ciphertext en bytes si besoin.
- Lancer la commande de chiffrement OpenSSL pour déchiffrer le Ciphertext avec le Password.

```
cmd = [
    "openssl", "enc",                   # Commande OpenSSL pour le chiffrement/déchiffrement symétrique
    "-d", f"-{cipher}",                 # Mode de déchiffrement et choix du chiffrement
    "-base64",                          # L'entrée est fournie en base64
    "-pbkdf2",                          # Utiliser PBKDF2 pour dériver la clé à partir de la passphrase
    "-pass", f"pass:{passphrase}",      # Transmet la passphrase à OpenSSL
    "-A"                                # Traiter le base64 comme une seule ligne
]
```

- Gère les erreurs OpenSSL.
- Récupère le Plaintext en octets.
- Soit le renvoie tel quel.
- Soit essaie de le décoder en UTF-8.
- Gère le cas où le résultat n'est pas du texte valide.

**FLAG 2 : Chiffrement symétrique**

**Objectif :**

- Chiffrer un Plaintext avec une clé publique.

**Méthode :**

- Récupérer la clé publique de « pki_tutorial ».
- Lancer le code « encrypt_plaintext_with_pk.py »
- Chiffrer le Plaintext « I got it » avec cette clé publique.

**Explications du code :**

- Stocker le Plaintext dans un fichier texte.
- Lancer la commande de chiffrement OpenSSL qui utilise la clé publique de pki_tutorial.

```
cmd = [
    "openssl", "pkeyutl",                                # Commande OpenSSL pour opérations sur clés publiques/privées
    "-encrypt",                                          # Je veux chiffrer
    "-pubin",                                            # La clé fournie est une clé publique
    "-inkey", "./Keys/PublicKeys/pk_pki_tutorial.pem",   # Chemin vers cette clé publique
    "-in", "./Texts/msg_pki_tutorial.txt",               # Fichier contenant le message en clair
    "-out", "./Binary/msg_pki_tutorial.bin"              # Fichier de sortie, contenant le ciphertext binaire
]
```

- Convertir la sortie en hexadécimal.

```
# Convertir en hexadécimal
cmd = [
    "xxd",
    "-p",
    "./Binary/msg_pki_tutorial.bin"
]
```

- Stocker la sortie dans un fichier .hex.

**FLAG 3 : Créer une paire de clé publique/privée RSA**

**Objectif :**

- Générer ma propre paire de clé RSA publique/privée.

**Méthode :**

- Lancer le code « generate_pk_sk_rsa.py ».
- Upload la clé publique obtenue sur notre carte étudiante.

**Explications du code :**

- Lancer la commande OpenSSL pour générer notre clé privée RSA.

```
cmd = [
    "openssl", "genpkey",                    # Commande OpenSSL pour générer une clé privée
    "-algorithm", "RSA",                     # La clé privée générée est de type RSA
    "-pkeyopt", "rsa_keygen_bits:2048",      # La clé RSA générée est de taille 2048 bits
    "-out", "./Keys/PrivateKeys/my_sk.pem"   # Fichier de sortie contenant la clé privée RSA générée
]
```

- Lancer la commande OpenSSL pour extraire notre clé publique depuis notre clé privée.

```
cmd = [
    "openssl", "pkey",                       # Commande OpenSSL pour lire la clé privée et en sortir la partie publique
    "-in", "./Keys/PrivateKeys/my_sk.pem",   # Chemin vers la clé privée
    "-pubout",                               # N'écrire que la clé publique mais pas la clé privée
    "-out", "./Keys/PublicKeys/my_pk.pem"    # Fichier de sortie contenant la clé publique RSA extraite de la clé privée
]
```

- Ma clé privée est dans ./Keys/PrivateKeys/my_sk.pem
- Ma clé publique est dans ./Keys/PublicKeys/my_pk.pem

**FLAG 4 : Signature RSA**

**Objectif :**

- Signer un message avec ma clé privée RSA.

**Méthode :**

- Lancer le code « sign_challenge_sk_rsa.py » pour signer le message avec ma clé privée RSA.

**Explications du code :**

- Lancer la commande OpenSSL pour signer le message avec ma clé privée RSA.

```
cmd = [
    "openssl", "dgst",                          # Commande OpenSSL pour calculer un condensat et signer
    "-sha256",                                  # Utiliser SHA-256 comme fonction de hachage
    "-sign", "./Keys/PrivateKeys/my_sk.pem",    # Clé privée utilisée pour signer
    "-out", "./Binary/sign_challenge1.bin",     # Fichier de sortie contenant la signature binaire
    "./Texts/sign_challenge1.txt"               # Fichier contenant le message à signer
]
```

- Convertir la sortie en héxadécimal.

```
cmd = [
    "xxd",
    "-p",
    "./Binary/sign_challenge1.bin"
]
```

- Stocker la sortie dans un fichier .hex.

**FLAG 5 : Attaque de dictionnaire**

**Objectif :**

- Trouver la clé privée d'un utilisateur par attaque sur dictionnaire.

**Contexte :**

- Je connais l'identifiant d'un utilisateur.
- J'ai un Plaintext que l'utilisateur a chiffré avec sa clé privée ainsi que le Ciphertext associé.
- J'ai une liste très grande contenant les mots de passe possible de cet utilisateur.

**Méthode :**

- Lancer le code « decrypt_ciphertext_brute_force.py » pour chiffrer le Plaintext avec chacune des clés de la liste des clés possibles et comparer avec le Ciphertext cible pour voir quelle clé privée signe bien le Plaintext.

**Explications du code :**

- Parcourir la liste des clés privées potentielles.
- Déchiffrer le Ciphertext avec chacune de ces clés.
- Comparer le Plaintext obtneu avec le Plaintext original jusqu'à trouver la bonne clé privée.

**FLAG 6 : Générateur de clé RSA**

**Objectif :**

- Générer une paire de clés RSA dont l'exposant public *e* est fabriqué à partir de la chaîne « +++ATRIUM+++ »

**Méthode :**

- Fabriquer un exposant RSA personnalisé à partir de « +++ATRIUM+++ ».
- Générer une clé privée RSA avec cet exposant via OpenSSL.
- En extraire la clé publique.
- Vérifier si la chaîne « +++ATRIUM+++ » apparaît dans la version Base64 de cette clé publique.

**Explication du code :**

- Décoder la cible de Base64 vers des octets binaires.

```
atrium_b64_dec = base64.b64decode(TARGET)
```

- Forcer un exposant impair en ajoutant l'octet b"\x01".

```
atrium_b64_dec += b"\x01"
```

- Convertir les octets en entier pour en faire l'exposant RSA et y ajouter un très grand nombre pour gonfler la valeur finale.

```
atrium_int = int.from_bytes(atrium_b64_dec, byteorder="big") + (256 * 10**100)
```

- Lancer la commande OpenSSL pour générer la clé privée.

```
print("Génération de la clé privée avec OpenSSL...")
cmd_keygen = [
    "openssl", "genpkey", # Lance OpenSSL et la sous-commande de génération de clé privée.
    "-algorithm", "RSA", # Demande une clé RSA.
    "-out", str(sk_path), # Fichier de sortie de la clé privée.
    "-outform", "PEM", # Format PEM:fichier texte encadré par des lignes "-----BEGIN ...-----".
    "-pkeyopt", f"rsa_keygen_pubexp:{atrium_int}", # Définit l'exposant public RSA e à la valeur cible.
    "-pkeyopt", "rsa_keygen_bits:2048", # Taille de la clé RSA : 2048 bits.
    "-pkeyopt", "rsa_keygen_primes:2", # Demande une RSA classique à deux nombres premiers p et q.
]
```

- Affichage du code de retour du processus, des erreurs, et de la sortie normale.
- Extraction de la clé publique depuis la clé privée.

```
cmd_pubout = [
    "openssl", "pkey", # Manipule une clé déjà existante.
    "-in", str(sk_path), # Lit la clé privée générée juste avant.
    "-pubout", # Extrait seulement la partie publique.
    "-out", str(pk_path), # Ecrit la clé publique dans un autre fichier.
]
```

- Découpe le texte ligne par ligne et reformate correctement la sortie.
- Teste si la chaîne « +++ATRIUM+++ » est présente comme sous-chaîne dans la Base64 de la clé publique.

**FLAG 7 : Connexion CHAP**

**Objectif :**

- M'authentifier avec le protocole CHAP.

**Méthode :**

- Id = AdrienPanguel
- Mdp = __CHAP__
- Lancer le code « decrypt_challenge_CHAP.py » qui utilise la fonction « decrypt() » du code « decrypt.py » pour déchiffrer le challenge donné.

**FLAG 8 : Connexion PK**

**Objectif :**

- M'authentifier avec le protocole à clef publique.

**Méthode :**

- Id = AdrienPanguel
- Mdp = __PK__
- Lancer le code « sign_challenge_PK_rsa.py » pour signer le challenge donné.

**FLAG 9 : Certificat**

**Objectif :**

- Fournir un certificat valide qui atteste officiellement que ma paire de clé RSA publique/privée m'appartient bien.

**Méthode :**

- Lancer le code « provide_certificate.py » pour générer un certificat pour ma paire de clé RSA publique/privée.

**Explications du code :**

- Lancer la commande OpenSSL pour générer un certificat pour ma paire de clé RSA publique/privée.

```
cmd = [
    "openssl", "req",                                       # Commande OpenSSL pour créer une requête de certificat
    "-new",                                                 # Demande la création d'une nouvelle CSR
    "-key", "./Keys/PrivateKeys/sk_atrium.pem",             # Clé privée dont on veut donner un certificat
    "-batch",                                               # Eviter les questions interactives
    "-subj", "/CN=AdrienPanguel",                           # Définir directement le sujet du certificat
    "-out", "./Certificates/certificate_atrium.csr.pem"     # Chemin du fichier de sortie
]
```

**FLAG 10 : Broadcast**

**Objectif :**

-

**Méthode :**

- Lancer le code « build_session_key.py » pour, étant donné un message clair choisi « AdrienPanguel123 », chiffrer chiffrer ce message avec la clé publique donnée, pour obtenir une session-key.
- Lancer le code « encrypt_plaintext_session_key.py » pour chiffrer un message « test » avec cette nouvelle clé pour obtenir un Ciphertext.

**Explications du code :**

- Lancer la commande OpenSSL pour chiffrer un message clair avec une clé publique donnée.

```
cmd = [
    "openssl", "pkeyutl",                                   # Commande OpenSSL pour chiffrer
    "-encrypt",                                             # Choix de chiffrer
    "-pubin",                                               # Dire que la clé fournie est la clé publique
    "-inkey", "./Keys/PublicKeys/pk_broadcast.pem"          # Clé publique utilisée pour chiffrer
]
```

- Convertir la sortie en héxadécimal.
- Puis dans le second code, on chiffre un message clair « test » avec le message clair « AdrienPangue123 ».
- On met dans un fichier .json la session-key obtenue et le ciphertext.


```
{
  "session-key":
  "b5a25a0fce193fe776d3154e1dbe964474392052918e81698b497c3087460f7045bafd1e34b12b350a40b2d8261b120833006ec544caa25e96ec2bec6e3762d12
  83210a73509289b7431ce18ef025c089a08298ca81ac223add2800e576f5dc28c1a282113e2c898cc7fcc9dafa7ef34ac7bdf34cea344ac607b55cfe5f79f1f66d
  f1e7b405d77e67a76c9652c0958658c121873743c0cc39062290b06c22e1a1738a975884d34a583dd8429c3e221000cf70a362982053634b442dc2bcc3fd63e1c4
  6a37add62b28486cac02de6a23938942949f1c1716e84e911ae92a8e34a56b0d783cbb1084012a47f48f67cd908fe86aed4cccb5830f924dd40059062b5",
  "ciphertext": "U2FsdGVkX19jqtLBLFRzg9ioEkZ7jL5MQcr4dfQsVoE=\n"
}
```


- On broadcast ce fichier .json.

**FLAG 11 : Signature collective**

**Objectif :**

- Faire signer ma clé publique par 2 camarades avec leur clé privée.

**Méthode :**

- Lancer le code « sign_friends.py » pour signer la clé publique d'un camarade avec ma clé privée.

Explicaitons du code :

- Lancer la commande OpenSSL pour signer une clé publique avec ma clé privée.

```
cmd = [
    "openssl", "dgst",                          # Outil OpenSSL pour calculer un condensat
    "-sha256",                                  # Algorithme de hachage utilisé
    "-sign", "./Keys/PrivateKeys/my_sk.pem",    # Clé privée utilisée pour signer
    "./Keys/PublicKeys/pk_friend1.pem"          # Clé publique à signer
]
```

- Convertir la sortie en héxadécimal.

**FLAG 12: Shared modulus Attack (RSA) avec fuite de  Private Key**

**Objectifs :**

-

**Contexte :**

- Je connais les clés publiques $(e_A, N_A)$ et $(e_B, N_B)$.
- J'ai accès à la valeur $d_A$ de l'individu A.

**Méthode :**

- Connaissant $e_A$ et $d_A$, je peux calculer :

$$e_A \cdot d_A \equiv 1 \; mod \; \varphi(N_A)$$

- En posant :

$$M = e_A \cdot d_A - 1$$

- On peut réécrire :

$$M = k \cdot \varphi(N_A) = ed - 1 = 2^t \cdot r, avec \; r \; impair$$

- On choisit ensuite un entier aléatoire g entre 2 et $(N_A - 2)$, puis on calcule :

$$x = g^r \, mod N_A$$

- Ensuite on regarde la suite :

$$x, x^2, x^{2^2}, \dots, x^{2^t}$$

- C'est-à-dire :

$$g^r, g^{2r}, \dots, g^{2^{tr}} \; mod \; N_A$$

- Comme M est un multiple de$\varphi(N_A)$, on a souvent :

$$g^M \equiv 1 \bmod N_A$$

- Donc, en remontant la chaîne des carrés, on cherche le moment où on passe d'une valeur non triviale à 1, dont le cas intéressant est :

$$\begin{cases} y^2 \equiv 1 \bmod N_A \\ y \neq \pm 1 \; mod \; N_A \end{cases}$$

$$\Rightarrow y^2 \equiv 1 \bmod N_A \Rightarrow y^2 - 1 \equiv 0 \; mod \; N_A \Rightarrow (y-1)(y+1) \equiv 0 \bmod N_A$$

$$\Rightarrow N_A | (\mathrm{y} - 1)(\mathrm{y} + 1)$$

- On sait que $y \neq \pm 1 \; mod \; N_A$, donc que y est une racine carrée non triviale de 1 modulo $N_A$, c'est-à-dire que comme $N_A$ ne divise pas $(y-1)$ tout seul ou $(y+1)$ tout seul, alors p divise l'un et q divise l'autre, dans un de ces cas par exemple :

$$\begin{cases} \gcd(y-1, N_A) = p \\ q = \dfrac{N_A}{p} \end{cases}$$

- On a retrouvé p et q !

**Explications du code :**

**FLAG 13 : Shared Modulus (RSA)**

**Objectif :**

- Retrouver le Plaintext message $m$ qui a été chiffré par deux individus avec la méthode RSA utilisant le même modulus N.

**Rappel (construction de la clé privée en RSA) :**

- Generate two large primes p and q with $\mathrm{p} \neq \mathrm{q}$.
- Compute:
$$\mathrm{N} = \mathrm{p} \times \mathrm{q}$$
- Compute Euler's totient:
$$\varphi(\mathrm{N}) = (\mathrm{p} - 1) \times (\mathrm{q} - 1)$$
- Choose a random public exponent e (common choice is $\mathrm{e} = 65537$) such that:
$$\begin{cases} 1 < \mathrm{e} < \varphi(\mathrm{N}) \\ \gcd(\mathrm{e}, \varphi(\mathrm{N})) = 1 \end{cases}$$
- Compute the unique integer d such that:
$$\begin{cases} 1 < \mathrm{d} < \varphi(\mathrm{N}) \\ \mathrm{d} \times \mathrm{e} \equiv 1 \bmod \varphi(\mathrm{N}) \end{cases}$$
- **The public key is (N, e).**
- **The private key is (d, p, q).**

**Contexte :**

- Je connais les clés publiques $(e_1, N_1)$ et $(e_2, N_2)$.
- Le module est le même : $N_1 = N_2 = N$.
- Je connais les chiffrés du même message $c_1$ et $c_2$.
- Le message m est tel que $0 \leq m < N$.

**Attaque :**

- Vérifier que $e_1$ et $e_2$ sont premier entre eux : $\gcd(e_1, e_2) = 1$.
- D'après le théorème de Bézout, comme $e_1$ et $e_2$ sont premier entre eux :
$$u \cdot e_1 + v \cdot e_2 = 1$$
- Mettre le chiffré $c_1$à la puissance $u$ et le chiffré $c_2$à la puissance $v$ :
$$\begin{cases} c_1^u = m^{e_1 u} \; mod \; N = m^{e_1 \cdot u} \; mod \; N \\ c_2^v = m^{e_2 v} \; mod \; N = m^{e_2 \cdot v} \; mod \; N \end{cases}$$
- Attention, comme soit a soit b est négatif (supposons $a < 0$), je ne peux pas calculer $c_1^a$ directement, donc je dois passer par l'inverse modulaire :
$$c_1^{-|a|} = (c_1^{-1})^{|a|} \; mod \; N$$
- Et l'inverse modulaire de $c_1$ existe car $gcd(c_1, N) = 1$, sinon on aurait
- factorisé N, ce qui serait encore mieux.
- Je multiplie ces 2 chiffrés $c_1^u$ et $c_2^v$ entre eux :
$$c_1^u \cdot c_2^v = m^{e_1 \cdot u} \cdot m^{e_2 \cdot v} \; mod \; N = m^{e_1 \cdot u + e_2 \cdot v} \; mod \; N = m \; mod \; N = m$$
- J'ai retrouvé le message m !

**Remarque :**

- Quand $e_1$et $e_2$ ne sont pas premiers entre eux, je pose :
$$d = \gcd(e_1, e_2) > 1$$
- Tel que :

$$\begin{cases} e_1 = d \cdot a \\ e_2 = d \cdot b \\ \gcd(a, b) = 1 \end{cases}$$

- Comme $a$ et $b$ sont premiers entre eux, il existe $(u, v)$ tels que :
$$u \cdot a + v \cdot b = 1$$
- Donc :
$$u \cdot e_1 + v \cdot e_2 = d$$
- À partir des chiffrés, on calcule :
$$X = c_1^u \cdot c_2^v \bmod N = m^{e_1 u} \cdot m^{e_2 v} \bmod N = m^{e_1 \cdot u + e_2 \cdot v} \; mod \; N = m^d \; mod \; N = m^d$$
- Il suffit de prendre la racine $d$-ième entière :
$$m = \sqrt[d]{X}$$
- J'ai retrouvé le message m !

**<u>Explications du code :</u>**

-

**FLAG 14 : Attaque par recherche exhaustive sur une seed trop petite**

**Objectif :**

- Montrer qu'un système qui utilise un chiffrement correct en apparence, peut être cassé si la clé et l'IV proviennent d'un secret minuscule.

**Méthode :**

- Essayer toutes les seeds possibles en dérivant pour chaque seed une clé K et un IV.
- Si l'IV dérivé ets identique à l'IV connu, alors la seed est la bonne, donc la clé K aussi.
- Utiliser K et l'IV pour déchiffrer le ciphertext donné.

**Contexte :**

- Une seed de 16 bits est générée, et à partir d'elle, on dérive une clé secrète de 128 bits et un IV de 128 bits connu, ce qui nous donne un Ciphertext connu.
- Il n'existe donc que 65536 seeds possibles (très petit).
- Autrement dit, le système a l'air de manipuler du 128 bits, mais sa vraie sécurité dépend en fait d'un secret de 16 bits seulement.

**Explications du code :**

- Nettoyer le Ciphertext cible et le transformer en octets binaires.
- Tester toutes les seeds en les dérivant sur 32 octets [Key : 0 à 15, IV : 16 à 31].

```
km = key_expansion(seed)
```

- Comparer chaque IV obtenu à l'IV cible pour déterminer la bonne seed.
- Déchiffrer le ciphertext cible avec la bonne clé couplée au bon IV.

```
pt = openssl_decrypt_aes128cbc(cipher_bytes, key_hex, iv_hex)
```

**FLAG 15 : Attaque de Haastard (petit exposant publique commun)**

**Objectif :**

- Retrouver le Plaintext message $m$ qui a été chiffré par trois individus avec la méthode RSA utilisant le même petit exposant $e$ (passer par le théorèe chinois des restes).

**Contexte :**

- Je connais les chiffrés de 3 individus :

$$\begin{cases} c_1 \equiv m^e \ mod \ N_1 \\ c_2 \equiv m^e \ mod \ N_2 \\ c_3 \equiv m^e \ mod \ N_3 \end{cases}$$

- Je connais les clés publiques $(e_1, N_1)$, $(e_2, N_2)$, et $(e_3, N_3)$.
- Le message m est tel que $0 \leq m < N$.

**Attaque :**

- Mathématiquement, je cherche un entier $X$ tel que :

$$\begin{cases} X \equiv c_1 \bmod N_1 \\ X \equiv c_2 \bmod N_2 \\ X \equiv c_3 \bmod N_3 \end{cases}$$

- Comme :

$$c_i \equiv m^e \bmod N_i$$

- Cela revient à chercher un $X$ vérifiant :

$$\begin{cases} X \equiv m^e \bmod N_1 \\ X \equiv m^e \bmod N_2 \\ X \equiv m^e \bmod N_3 \end{cases}$$

- Le théorème chinois des restes dit que si $N_1$, $N_2$ et $N_3$ sont deux à deux premiers entre eux, alors ce système admet une solution unique modulo $M = N_1 N_2 N_3$.
- Autrement dit, on peut reconstruire un entier $X$ qui satisfait les trois congruences tel que :

$$0 \leq X < M$$

- Donc :

$$X \equiv m^e \bmod N_1 N_2 N_3 \equiv m^e \ mod \ M$$

- Pour calculer ce $X$, on pose :

$$\begin{cases} M = N_1 N_2 N_3 \\ M_i = \dfrac{M}{N_i} \\ u_i \equiv M_i^{-1} \bmod N_i \end{cases}$$

- D'où la formule explicite pour calculer $X$ :

$$X \equiv c_1 M_1 u_1 + c_2 M_2 u_2 + c_3 M_3 u_3 \bmod M = m^e \ mod \ M = m^e$$

- Je peux alors calculer :

$$m = \sqrt[e]{X}$$

- J'ai retrouvé le message m !

**Preuve de la validité du théorème chinois des restes :**

- A partir du système de congruences suivant :

$$\begin{cases} X \equiv m^e \bmod N_1 \\ X \equiv m^e \bmod N_2 \\ X \equiv m^e \bmod N_3 \end{cases} \Rightarrow \begin{cases} (X - m^e) \equiv N_1 \\ (X - m^e) \equiv N_2 \\ (X - m^e) \equiv N_3 \end{cases}$$

- Posons :

$$z = X - m^e$$

- On sait :

$$\begin{cases} N_1 \mid z \\ N_2 \mid z \\ N_3 \mid z \end{cases}$$

- Je veux montrer que :

$$N_1 N_2 N_3 \mid z$$

- **Étape 1 : combiner $N_1$ et $N_2$**
- Comme $N_1 \mid z$, il existe $a \in \mathbb{Z}$ tel que :

$$z = N_1 a$$

- Mais comme aussi $N_2 \mid z$, donc :

$$N_2 \mid N_1 a \Rightarrow N_2 | N_1 \; ou \; N_2 | a$$

- Or :

$$\gcd(N_1, N_2) = 1$$

- Donc $N_2$ ne divise pas $N_1$, donc :

$$N_2 \mid a$$

- Donc il existe $b \in \mathbb{Z}$ tel que :

$$a = N_2 b$$

- En remplaçant dans $z = N_1 a$, j'obtiens :

$$z = N_1 N_2 b$$

- Donc :

$$N_1 N_2 \mid z$$

- **Étape 2 : ajouter $N_3$**
- J'ai maintenant :

$$\begin{cases} N_1 N_2 \mid z \\ N_3 \mid z \end{cases}$$

- Je refais exactement le même raisonnement, comme $N_3 \mid z$, il existe $c \in \mathbb{Z}$ tel que :

$$z = N_3 c$$

- Mais comme aussi $N_1 N_2 \mid z$, donc :

$$N_1 N_2 \mid N_3 c \Rightarrow N_1 N_2 | N_3 \; ou \; N_1 N_2 | c$$

- Or $N_1, N_2, N_3$ sont deux à deux premiers entre eux, donc $N_1 N_2$ ne divise pas $N_3$, alors :

$$N_1 N_2 \mid c$$

- Donc il existe $d \in \mathbb{Z}$ tel que :

$$c = N_1 N_2 d$$

- En remplaçant dans $z = N_3 c$, j'obtiens :

$$z = N_1 N_2 N_3 d$$

- Donc :

$$N_1 N_2 N_3 \mid z$$

- Donc :

$$N_1 N_2 N_3 \mid (X - m^e)$$

- Autrement dit :

$$X \equiv m^e \bmod N_1 N_2 N_3$$

**FLAG 16 : RSA blinding**

**Objectif :**

- Obtenir sans autorisation une signature du directeur sur un message précis.

**Méthode :**

- Le serveur du directeur ne fait pas une vraie signature RSA complète, c'est-à-dire qu'il se comporte comme un oracle RSA brut, car il prend juste un entier en entrée et calcule $m^d mod N$, or RSA brut est malléable car la signature d'un produit est liée au produit des signatures.
- On commence par construire correctement le bloc $M$ correspondant au message que je veux faire signer avec la signature du directeur, avec l'encodage PKCS#1 v1.5 attendu par OpenSSL.
- Ensuite, au lieu d'envoyer directement $M$, je choisis un entier aléatoire $x$ inversible modulo $N$, et $e$ du directeur, j'envoie :

$$M' = M \cdot x^e \ mod N$$

- Le serveur signe alors cette valeur masquée et me renvoie :

$$(M')^d = (M \cdot x^e)^d \equiv M^d \cdot x \bmod N$$

- On obtient donc la vraie signature $M^d$, mais multipliée par le facteur $x$.
- Comme on connaît $x$, on calcule son inverse modulo $N$, puis on retire le masque :

$$S = (M')^d \cdot x^{-1} \equiv M^d \bmod N$$

- Le résultat $S$ est alors exactement la signature RSA valide du message voulu.

**Explications du code :**

- Récupère le modulus de pk_director.pem avec OpenSSL.
- Récupère le $N$ de la clé publique du directeur.

```
N_hex = result.stdout.replace("Modulus=", "").strip().lower()
```

- Convertit le résultat en minuscules.
- Construit le bloc PKCS#1 v1.5.

```
msg = b"I, the lab director, hereby grant AdrienPanguel permission to take the BiblioDrone-NG."
msg_hash = hashlib.sha256(msg).digest()
```

- Calcule le message aveuglé.

```
x = 2
M_blind = (M * pow(x, e, N)) % N
```

- Affiche la donnée hexadécimale à envoyer au directeur.

**FLAG 17 : Interpolation polynomiale**

**Objectifs :**

- Retrouver les coefficients d'un polynôme P(X) de degré 8, qui a donc 9 coefficients inconnus, à partir de 9 couple points/images.

**Méthode :**

- Certaines firmware-keys sont produites par un polynôme, donc connaissant 9 firmware-keys, comme ce polynôme est de degré 8, alors il possède 9 coefficients inconnus, je peux le reconstruire.
- Les 9 premières clés connues te donnent ces 9 points, car pour chaque entrée $x_i$, tu connais la valeur correspondante $P(x_i)$. Avec ces 9 couples $(x_i, P(x_i))$, ton script fait une interpolation polynomiale et reconstitue entièrement le polynôme $R(X)$.
- Une fois le polynôme retrouvé, tu peux l'évaluer en un point particulier, ici $X = 0$. Le script calcule donc :

$$R(0) = 3227412088583349083$$

- Puis l'écrit en hexadécimal :

0x2cca1231ebcfe35b

- Cette valeur correspond à la firmware update key attendue par la borne. On enlève simplement le préfixe 0x pour saisir :

$$2cca1231ebcfe35b$$

**Explications du code :**

- Découper chaque MAC en 4 blocs X, A, B, C pour chaque flag.
- Lancer l'algorithme de Lagrange pour chacun de ces points